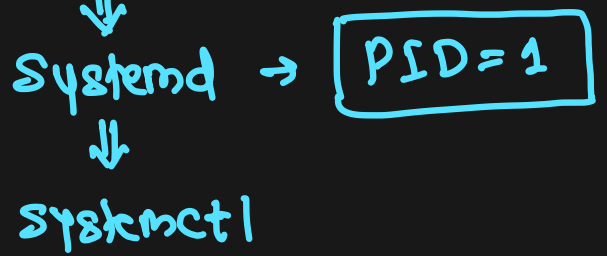
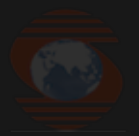
Service Management



Services and Daemons



- While the terms service and daemon are sometimes used interchangeably, there are specific differences between the two
- Daemons are noninteractive programs running on the system. They do not provide direct communication with users and operate in the background.
- Services are interactive, though they too operate in the background. Users may interact with services. Services are the purpose behind server computers.
- Note that services are usually implemented as daemons and also contain an interactive component. Hence, services such as SSH will have a related daemon, such as sshd.
- Service and daemon management is an essential part of the sysadmin role
- Here are a few critical services that users rely on throughout the business day
  - Web service: Makes web pages available across the network and relies on httpd
  - SSH service: Allows secure remote administration and relies on sshd
  - NFS service: Allows remote access to exported directories and relies on several daemons, including nfsd and mountd
  - NTP service: Synchronizes time among network servers and relies on ntpd

SSH service → sshd

Web server service → httpd

System Initialization

- System initialization is the process that begins when the kernel first loads → booting process
- It involves the loading of the operating system and its various components, including the boot process
- System initialization is carried out by an init daemon in Linux—the "parent of all processes."
- The init daemon refers to a configuration file and initiates the processes listed in it
- This prepares the system to run the required software
- The init daemon runs continuously until the system is powered off, and programs on the system will not run without it
- On Linux, there are two main methods that initialize a system: systemd and SysVinit. Which method is active on your Linux system will affect how you manage services on that system
- Most modern Linux distributions use systemd to manage initialization
- However, some distributions, especially older ones, rely on a different initialization mechanism named System V

deprecated

systemd → has PID = 1

- The systemd software suite provides an init method for initializing a system
- It also provides tools for managing services on the system that derive from the init daemon
- The systemd suite is now the dominant init method in modern Linux distributions, and was designed as a replacement for other methods like SysVinit
- The systemd suite offers several improvements over older methods
- For example, it supports parallelization (starting programs at the same time for quicker boot) and reduces shell overhead
- In systemd, Control Groups (cgroups) are used to track processes instead of process IDs (PIDs), which provides better isolation and categorization for processes

systemctl command

- One of the most frequent tasks for administrators is managing services and daemons
- Management includes starting, stopping, and checking the status of these applications
- It's critical to remember that Linux services and daemons acquire their settings from configuration files that are read when the system or the applications start
- Any changes to these settings occur by editing the files, which must then be reread to discover the new configurations
- In addition, administrators may choose whether a given service starts when the system boots
- Managing Services with systemctl
  - start: Start the service or daemon (not persistent)
  - stop: Stop the service or daemon (not persistent)
  - restart: Restart the service or daemon
  - enable: Set the service or daemon to start at boot (persistent)
  - disable: Set the service or daemon not to start at boot (persistent)
  - status: Display the current status of the service or daemon

systemd unit files

- Under the systemd initialization process, the term unit refers to a resource that systemd knows how to manage via daemons and configuration files
- This system is much more flexible than earlier initialization methods
- Unit files define how systemd manages the unit
- There are many systemd unit file types, with .service, .timer, .mount, and .target as the primary examples
- Stored in:
  - /etc/systemd/system: custom
  - /usr/lib/systemd/system: default
- Commands
  - Use **systemctl –t help** to get list of service units
  - Use **systemctl list-unit-files** for listing unit files
  - Use **systemctl list-units --type=service** for filtering units with type service

Unit file types

- .service: Background services
- .socket: IPC/network sockets
- .target: Group of units (like runlevels)
- .mount: Filesystem mounts
- .automount: On-demand mounts
- .timer: Scheduled jobs
- .path: File-based triggers
- .device: Hardware devices
- .swap: Swap partitions

System Targets

- A target in systemd is a group of units (services, mounts, etc.) that represents a system state
- A mode of operation (CLI mode, GUI mode, rescue mode)
- A replacement for traditional runlevels

Run level	Target	Description
0	poweroff.target	Shutdown the system
1	rescue.target	Single user mode
2	multi-user.target	Multi-user without networking
3	multi-user.target	CLI (without GUI) with networking
4	multi-user.target	Reserved
5	graphical.target	With GUI mode
6	reboot.target	Reboot the system

- Check current target: sudo systemctl get-default
- Set a target: sudo systemctl set-default <target>

Modifying systemd Service configurations

- Default system provided unit files are located in /usr/lib/systemd/system
- Custom unit files are located in /etc/systemd/system
- Run-time automatically generated unit files are located in /run/systemd
- While modifying a unit file, do NOT edit the file in /usr/lib/systemd/system, instead create a custom file in /etc/system/system that is used as an overlay file
- Better option is to use **systemctl edit unit.service** command to edit unit files
- Use **systemctl show** command to show the available parameters
- Use after modifying the parameters

Service Unit File Structure

- Example: /etc/systemd/system/myapp.service

```
[Unit]
description=My application
After=network.target

[Service]
ExecStart=/usr/bin/python3 /home/vagrant/myapp.py
Restart=always
User=vagrant
```

- Reload the daemon
  - sudo systemctl daemon-reload
- Debug the service
  - sudo journalctl –u myapp

Mount Unit File Structure

- Example: /etc/systemd/system/mydisk.mount

```
[Unit]
Description=auto mount mydisk
After=local-fs.target

[Mount]
What=/dev/nvme0n2p1
Where=/mydisk
Type=ext4
Options=defaults

[Install]
WantedBy=multi-user.target
```